

Discrete Rolling Reserve Policies

A self-contained simulation study

Sebastián Calcetero Vanegas
Independent Researcher, Toronto, Ontario, Canada

September 2026

Contents

1 Purpose and design	1
2 Predictive environment	3
3 Nominal rolling policies	5
3.1 Frictionless rule	5
3.2 Quadratic proportional-adjustment rule	5
3.3 Absolute-value rule	6
4 Performance measures	7
4.1 Nominal reserve paths	10
5 KL-robust counterparts	12
5.1 Robust reserve paths	21
6 Predictive-dispersion sensitivity	23
7 Numerical verification	25
8 Conclusion	25

1 Purpose and design

The paper treats reserve selection as a decision problem layered on a predictive reserving model. At weekly review n , that model supplies the conditional law F_n of the outstanding liability L_n . Given the previously carried reserve R_{n-1} , the insurer selects the new reserve from the rolling objective

$$R_n^* = \arg \min_{r > 0} \{c_n^o \mathbb{E}_n[(r - L_n)^+] + c_n^u \mathbb{E}_n[(L_n - r)^+] + \Psi_n(r, R_{n-1})\}.$$

Here, c_n^o and c_n^u are the reduced-form marginal costs of excess reserve and reserve shortfall, respectively, while Ψ_n assigns a cost to revising the reserve. The paper studies three choices:

$$\begin{aligned}
\text{frictionless: } \Psi_n(r, R_{n-1}) &= 0, \\
\text{quadratic proportional: } \Psi_n(r, R_{n-1}) &= \eta_n \left(\frac{r - R_{n-1}}{R_{n-1}} \right)^2, \\
\text{absolute value: } \Psi_n(r, R_{n-1}) &= \eta_n |r - R_{n-1}|.
\end{aligned}$$

The first choice produces the conditional quantile $F_n^{-1}\{c_n^u/(c_n^o + c_n^u)\}$. The quadratic policy moves gradually toward that contemporaneous benchmark, whereas the absolute-value policy leaves the reserve unchanged inside an endogenous no-action band and moves it to the nearest boundary otherwise. The code below implements these objectives at each discrete review date.

This report reproduces the controlled simulation study for the discrete rolling reserve policies. It is deliberately self-contained: the predictive distributions, policy functions, robust solvers, performance measures, figures, and checks are all generated by the code below. No fitted data object or external simulation output is read.

There are $N = 52$ weekly review dates. At review n , the liability has a known Gamma predictive distribution with mean m_n and coefficient of variation v_n :

$$L_n \mid \mathcal{F}_{t_n} \sim \text{Gamma}(\alpha_n, \beta_n), \quad \alpha_n = v_n^{-2}, \quad \beta_n = m_n v_n^2,$$

where β_n is the scale parameter. The experiment is not intended to establish empirical superiority. Its purpose is to make the policy mechanisms observable under a known law and to measure the adequacy cost of reserve stability. We use $c^o = 1$ and $c^u = 3$. These are reduced-form marginal decision costs, so the frictionless critical probability is

$$q = \frac{c^u}{c^o + c^u} = 0.75.$$

The corresponding buffer is therefore generated by the cost asymmetry rather than imposed as an external VaR constraint.

```

# Global experiment settings. Only base R and knitr are required.
N <- 52L
c_over <- 1
c_under <- 3
C_cost <- c_over + c_under
q_star <- c_under / C_cost

# Three revision-cost configurations. "Balanced" is a central reference,
# not an estimated optimum or a universally recommended calibration.
scenario_spec <- data.frame(
  Configuration = c("Reactive", "Balanced", "Stable"),
  QuadraticLambda = c(0.10, 1.50, 5.00),
  EtaAbsolute = c(0.10, 0.42, 0.62),
  stringsAsFactors = FALSE
)

# KL radii are studied separately while balanced revision costs are fixed.
ambiguity_spec <- data.frame(
  Ambiguity = c("Small", "Moderate", "Large"),
  Epsilon = c(0.005, 0.080, 0.250),
  stringsAsFactors = FALSE
)

stopifnot(
  q_star > 0,
  q_star < 1,
  all(q_star - scenario_spec$EtaAbsolute / C_cost > 0),

```

```
all(q_star + scenario_spec$EtaAbsolute / C_cost < 1)
)
```

2 Predictive environment

The mean and CV sequences are evaluated only at the 53 displayed grid points $n = 0, \dots, 52$. The four quarters create distinct environments: a relatively stable opening, an oscillatory transition, a pronounced middle-year reversal, and a persistent late increase. The policies never interpolate between weekly reviews.

For transparency, the deterministic functions used to construct this sequence are stated explicitly. Define

$$S(x; a, b) = \frac{1}{2} \left\{ 1 + \tanh \left(\frac{x - a}{b} \right) \right\}, \quad w(x) = S(x; 0.27, 0.035) \{1 - S(x; 0.66, 0.05)\}.$$

The predictive mean is

$$\begin{aligned} m(x) = & 100 + 8\{1 - S(x; 0.22, 0.07)\} + 1.2 \sin(14\pi x) \\ & + 0.7 \cos(24\pi x + 0.25) + 0.45 \sin(34\pi x + 0.60) \\ & + w(x) [8.5 \sin\{8\pi(x - 0.25)\} + 3.2 \sin\{18\pi(x - 0.25)\}] \\ & + 3.2 S(x; 0.62, 0.045) \{1 - S(x; 0.78, 0.05)\} \\ & + 24 S(x; 0.73, 0.05) \frac{x - 0.73}{0.27}, \end{aligned}$$

and its coefficient of variation is

$$\begin{aligned} v(x) = \max \bigg\{ & 0.045, 0.060 + 0.100 S(x; 0.25, 0.035) \{1 - S(x; 0.66, 0.05)\} \\ & + 0.030 S(x; 0.72, 0.05) + 0.008 \sin(10\pi x + 0.4) + 0.003 \cos(18\pi x) \bigg\}. \end{aligned}$$

The experiment evaluates $m_n = m(x_n)$ and $v_n = v(x_n)$ at $x_n = n/N$, so these formulas define the complete Gamma predictive law at every weekly review.

Smooth transition used to construct the deterministic mean and CV sequences.

```
S <- function(t, center, width) {
  0.5 * (1 + tanh((t - center) / width))
}

mean_liability <- function(t) {
  early_level <- 8 * (1 - S(t, 0.22, 0.07))
  small_oscillation <-
    1.20 * sin(14 * pi * t) +
    0.70 * cos(24 * pi * t + 0.25) +
    0.45 * sin(34 * pi * t + 0.60)

  middle_window <- S(t, 0.27, 0.035) * (1 - S(t, 0.66, 0.05))
  middle_movement <- middle_window * (
    8.5 * sin(8 * pi * (t - 0.25)) +
    3.2 * sin(18 * pi * (t - 0.25))
  )

  recovery <-
    3.2 * S(t, 0.62, 0.045) * (1 - S(t, 0.78, 0.05))
  late_trend <- S(t, 0.73, 0.05) * 24 * (t - 0.73) / 0.27
}
```

```

100 + early_level + small_oscillation + middle_movement +
  recovery + late_trend
}

liability_cv <- function(t) {
  middle_window <- S(t, 0.25, 0.035) * (1 - S(t, 0.66, 0.05))
  late_level <- S(t, 0.72, 0.05)
  local_movement <-
    0.008 * sin(10 * pi * t + 0.4) +
    0.003 * cos(18 * pi * t)

  pmax(
    0.060 + 0.100 * middle_window + 0.030 * late_level +
      local_movement,
    0.045
  )
}

make_input <- function(cv_multiplier = 1) {
  week <- 0:N
  time <- week / N
  mean <- mean_liability(time)
  cv <- cv_multiplier * liability_cv(time)
  shape <- 1 / cv^2
  scale <- mean * cv^2

  list(
    week = week,
    time = time,
    decision = 2:length(week),
    mean = mean,
    cv = cv,
    shape = shape,
    scale = scale,
    frictionless = qgamma(q_star, shape = shape, scale = scale),
    cv_multiplier = cv_multiplier
  )
}

input <- make_input()
input$initial <- input$frictionless[1L]

```

The next helpers evaluate the predictive CDF, its inverse, and the exact expected asymmetric adequacy cost. For a Gamma variable, the truncated first moment is available through a Gamma CDF with shape increased by one; Monte Carlo is not needed for the performance calculations.

```

cdf_at <- function(reserve, input) {
  pgamma(reserve, shape = input$shape, scale = input$scale)
}

quantile_at <- function(probability, input) {
  qgamma(probability, shape = input$shape, scale = input$scale)
}

expected_adequacy <- function(reserve, input) {
  probability <- cdf_at(reserve, input)

```

```

truncated_mean <- input$mean * pgamma(
  reserve,
  shape = input$shape + 1,
  scale = input$scale
)

expected_over <- reserve * probability - truncated_mean
expected_under <-
  input$mean - truncated_mean - reserve * (1 - probability)

c_over * expected_over + c_under * expected_under
}

# One smooth, serially correlated Gamma-marginal realization is included only
# as a visual scale reference. It is not used to calculate any policy or metric.
simulate_gamma_path <- function(input, seed = 13782L, rho = 0.98) {
  set.seed(seed)
  latent <- numeric(length(input$week))
  latent[1L] <- rnorm(1L)
  innovations <- rnorm(length(input$week) - 1L)

  for (i in 2:length(latent)) {
    latent[i] <- rho * latent[i - 1L] +
      sqrt(1 - rho^2) * innovations[i - 1L]
  }

  probabilities <- pmin(pmax(pnorm(latent), 1e-10), 1 - 1e-10)
  qgamma(
    probabilities,
    shape = input$shape,
    scale = input$scale
  )
}

simulated_liability <- simulate_gamma_path(input)

```

3 Nominal rolling policies

3.1 Frictionless rule

Without revision costs, the reserve at week n is simply

$$Q_n = F_n^{-1}(q).$$

This is already stored as `input$frictionless`.

3.2 Quadratic proportional-adjustment rule

For a quadratic penalty, we set

$$\eta^{(2)} = \lambda(c^o + c^u)R_0.$$

Given the previous reserve, the new reserve is the scalar root of

$$(c^o + c^u)F_n(r) - c^u + 2\eta^{(2)}\frac{r - R_{n-1}}{R_{n-1}^2} = 0.$$

Convexity places the root between the previous reserve and the current frictionless reserve. This gives a stable numerical bracket and makes the code particularly transparent.

```
quadratic_policy <- function(lambda, input) {
  eta <- lambda * C_cost * input$initial
  reserve <- numeric(length(input$week))
  reserve[1L] <- input$initial
  residual_at_solution <- numeric(length(input$week))

  for (i in input$decision) {
    previous <- reserve[i - 1L]
    frictionless <- input$frictionless[i]

    # First-order condition at the current week.
    residual <- function(r) {
      C_cost * pgamma(
        r,
        shape = input$shape[i],
        scale = input$scale[i]
      ) - c_under +
        2 * eta * (r - previous) / previous^2
    }

    bracket <- sort(c(previous, frictionless))
    if (diff(bracket) <= 1e-12 * max(1, bracket[2L])) {
      reserve[i] <- frictionless
    } else {
      reserve[i] <- uniroot(
        residual,
        interval = bracket,
        tol = 1e-11
      )$root
    }
    residual_at_solution[i] <- residual(reserve[i])
  }

  list(
    reserve = reserve,
    eta = eta,
    max_foc_error = max(
      abs(residual_at_solution[input$decision])
    )
  )
}
```

3.3 Absolute-value rule

For a penalty $\eta^{(1)}|r - R_{n-1}|$, define

$$A_n = F_n^{-1}\left(q - \frac{\eta^{(1)}}{c^o + c^u}\right), \quad B_n = F_n^{-1}\left(q + \frac{\eta^{(1)}}{c^o + c^u}\right).$$

The previous reserve is kept when it lies inside $[A_n, B_n]$; otherwise it is projected to the nearest boundary. The band is symmetric in probability space, but it need not be symmetric in reserve units.

```
absolute_policy <- function(eta, input) {
  lower_probability <- q_star - eta / C_cost
  upper_probability <- q_star + eta / C_cost
  lower <- quantile_at(lower_probability, input)
  upper <- quantile_at(upper_probability, input)

  reserve <- numeric(length(input$week))
  reserve[1L] <- input$initial
  action <- rep("Initial", length(input$week))

  for (i in input$decision) {
    previous <- reserve[i - 1L]

    if (previous < lower[i]) {
      reserve[i] <- lower[i]
      action[i] <- "Increase"
    } else if (previous > upper[i]) {
      reserve[i] <- upper[i]
      action[i] <- "Decrease"
    } else {
      reserve[i] <- previous
      action[i] <- "Keep"
    }
  }

  list(
    reserve = reserve,
    lower = lower,
    upper = upper,
    action = action
  )
}
```

4 Performance measures

The predictive mean is the operational point-estimate comparator. The main measures are:

- **adequacy regret:** adequacy cost relative to the frictionless minimum;
- **adequacy gain:** adequacy-cost reduction relative to booking the mean;
- **attained percentile:** $100F_n(R_n)$, averaged over the 52 decisions;
- **movement reduction:** reduction in total variation relative to the mean path;
- **inaction frequency:** fraction of exact keeps under the absolute-value rule.

Revision penalties are intentionally excluded from adequacy regret. Regret is the adequacy price of stability, not a comparison of objectives that use different revision weights.

```
total_movement <- function(reserve) {
  sum(abs(diff(reserve)))
}

evaluate_path <- function(
  reserve,
```

```

    input,
    configuration,
    policy,
    action = NULL) {

  d <- input$decision
  mean_path <- input$mean
  mean_path[1L] <- input$initial

  adequacy_cost <- sum(expected_adequacy(reserve, input)[d])
  optimum_cost <- sum(
    expected_adequacy(input$frictionless, input)[d]
  )
  mean_cost <- sum(expected_adequacy(mean_path, input)[d])
  attained_percentile <- cdf_at(reserve, input)[d]

  movement <- total_movement(reserve)
  mean_movement <- total_movement(mean_path)

  data.frame(
    Configuration = configuration,
    Policy = policy,
    RegretPct = 100 * (adequacy_cost / optimum_cost - 1),
    GainVsMeanPct = 100 * (1 - adequacy_cost / mean_cost),
    AveragePercentile = 100 * mean(attained_percentile),
    ProbabilityDisplacement =
      100 * mean(abs(attained_percentile - q_star)),
    BufferVsMeanPct =
      100 * mean((reserve[d] - mean_path[d]) / mean_path[d]),
    TotalMovement = movement,
    MovementReductionVsMeanPct =
      100 * (1 - movement / mean_movement),
    InactionPct = if (is.null(action)) {
      NA_real_
    } else {
      100 * mean(action[d] == "Keep")
    },
    stringsAsFactors = FALSE
  )
}

```

The following block runs the three configurations and retains both paths and diagnostics.

```

nominal_paths <- list()
nominal_metrics <- list()
nominal_diagnostics <- list()

mean_path <- input$mean
mean_path[1L] <- input$initial
nominal_metrics[[1L]] <- evaluate_path(
  mean_path,
  input,
  "Benchmark",
  "Predictive mean"
)
nominal_metrics[[2L]] <- evaluate_path(

```

```

input$frictionless,
input,
"Benchmark",
"Frictionless reserve"
)

for (j in seq_len(nrow(scenario_spec))) {
  specification <- scenario_spec[j, ]
  quadratic <- quadratic_policy(
    specification$QuadraticLambda,
    input
  )
  absolute <- absolute_policy(
    specification$EtaAbsolute,
    input
  )

  nominal_paths[[specification$Configuration]] <- data.frame(
    Week = input$week,
    Mean = input$mean,
    Simulated = simulated_liability,
    Frictionless = input$frictionless,
    Quadratic = quadratic$reserve,
    Absolute = absolute$reserve,
    BandLower = absolute$lower,
    BandUpper = absolute$upper,
    AbsoluteAction = absolute$action,
    stringsAsFactors = FALSE
  )

  nominal_metrics[[length(nominal_metrics) + 1L]] <- evaluate_path(
    quadratic$reserve,
    input,
    specification$Configuration,
    "Quadratic penalty"
  )
  nominal_metrics[[length(nominal_metrics) + 1L]] <- evaluate_path(
    absolute$reserve,
    input,
    specification$Configuration,
    "Absolute-value penalty",
    action = absolute$action
  )

  nominal_diagnostics[[j]] <- data.frame(
    Configuration = specification$Configuration,
    MaxQuadraticFOCError = quadratic$max_foc_error,
    stringsAsFactors = FALSE
  )
}

nominal_metrics <- do.call(rbind, nominal_metrics)
nominal_diagnostics <- do.call(rbind, nominal_diagnostics)
rownames(nominal_metrics) <- NULL

```

Table 1: Revision-cost configurations.

Configuration	Quadratic lambda	Absolute eta	No-action probability band
Reactive	0.1	0.10	[0.7250, 0.7750]
Balanced	1.5	0.42	[0.6450, 0.8550]
Stable	5.0	0.62	[0.5950, 0.9050]

Table 2: Exact known-law performance of the nominal policies.

Configuration	Policy	Regret (%)	Gain vs mean (%)	Avg. percentile	Movement reduction (%)	Inaction (%)
Benchmark	Predictive mean	22.27	0.00	51.46	0.00	NA
Benchmark	Frictionless reserve	0.00	18.22	75.00	-4.24	NA
Reactive	Quadratic penalty	0.02	18.20	74.93	-1.55	NA
Reactive	Absolute-value penalty	0.29	17.98	74.47	11.64	19.23
Balanced	Quadratic penalty	1.49	17.00	74.19	26.34	NA
Balanced	Absolute-value penalty	4.05	14.90	73.46	50.02	44.23
Stable	Quadratic penalty	4.75	14.33	72.93	57.57	NA
Stable	Absolute-value penalty	7.08	12.43	74.23	73.17	69.23

The predictive mean occupies an average percentile of 51.46%, whereas the frictionless reserve attains 75% by construction. Relative to booking the mean, the balanced quadratic rule reduces movement by 26.34% at only 1.49% adequacy regret. The balanced absolute-value rule reduces movement by 50.02%, keeps the reserve unchanged in 44.23% of weekly decisions, and incurs 4.05% regret. The stable specification is visibly different: it suppresses more movement and pays a larger adequacy cost.

4.1 Nominal reserve paths

```

plot_colors <- c(
  mean = "#7A7A7A",
  simulated = "#1A1A1A",
  frictionless = "#277DA1",
  quadratic = "#E76F51",
  absolute = "#198754",
  band = "#2A9D8F",
  initial = "#6F42C1"
)

all_nominal_values <- unlist(lapply(
  nominal_paths,
  function(x) unlist(x[c(
    "Mean", "Simulated", "Frictionless", "Quadratic",
    "Absolute", "BandLower", "BandUpper"
  )])
))
nominal_limits <- range(all_nominal_values)
nominal_limits <- nominal_limits + c(-0.05, 0.05) * diff(nominal_limits)

draw_nominal_panel <- function(configuration) {

```

```

x <- nominal_paths[[configuration]]
plot(
  x$Week,
  x$Frictionless,
  type = "n",
  xlim = c(0, N),
  ylim = nominal_limits,
  xlab = "Weekly review n",
  ylab = "Reserve or liability",
  las = 1
)
abline(h = pretty(nominal_limits), col = "#E4E4E4", lwd = 0.7)
abline(v = c(13, 26, 39), col = "#999999", lty = 3)

polygon(
  c(x$Week, rev(x$Week)),
  c(x$BandLower, rev(x$BandUpper)),
  col = adjustcolor(plot_colors["band"], alpha.f = 0.14),
  border = NA
)
lines(
  x$Week, x$Simulated,
  col = plot_colors["simulated"], lwd = 2.6
)
points(
  x$Week, x$Simulated,
  col = plot_colors["simulated"], pch = 16, cex = 0.58
)
lines(x$Week, x$Mean, col = plot_colors["mean"], lty = 3, lwd = 1.8)
points(
  x$Week, x$Mean,
  col = plot_colors["mean"], pch = 1, cex = 0.62
)
lines(
  x$Week,
  x$Frictionless,
  col = plot_colors["frictionless"],
  lty = 2,
  lwd = 2.2
)
points(
  x$Week, x$Frictionless,
  col = plot_colors["frictionless"], pch = 17, cex = 0.66
)
lines(x$Week, x$Quadratic, col = plot_colors["quadratic"], lwd = 2.3)
points(
  x$Week, x$Quadratic,
  col = plot_colors["quadratic"], pch = 15, cex = 0.62
)
lines(x$Week, x$Absolute, col = plot_colors["absolute"], lwd = 2.4)
points(
  x$Week, x$Absolute,
  col = plot_colors["absolute"], pch = 18, cex = 0.74
)
points(

```

```

0, input$initial, pch = 8,
col = plot_colors["initial"], lwd = 2, cex = 1.05
)
keep <- x$AbsoluteAction == "Keep" & x$Week > 0
if (any(keep)) {
  points(
    x$Week[keep], x$Absolute[keep],
    pch = 21, bg = "white", col = plot_colors["absolute"],
    lwd = 1.8, cex = 0.92
  )
}
mtext(configuration, side = 3, line = 0.25, font = 2)
}

```

5 KL-robust counterparts

Robustness is varied independently of the revision penalties. For a proposed reserve r , the least-favourable density is proportional to

$$\exp\{\theta a_n(L_n; r)\} dP_n, \quad a_n(L_n; r) = c^o(r - L_n)^+ + c^u(L_n - r)^+.$$

The reserve and the exponential tilt are solved jointly subject to the binding KL equation. For the quadratic rule, the adverse CDF replaces the nominal CDF in the marginal first-order condition. For the absolute-value rule, the lower and upper boundaries are each solved with their own adverse tilt. The resulting least-favourable distribution is a reserve-dependent piecewise exponential tilt of the Gamma reference law; it is not itself a single Gamma distribution.

Because Gamma scale is proportional to the mean, the robust calculations can be performed with L_n/m_n , r/m_n , and the dimensionless tilt θm_n . This improves numerical conditioning without changing the KL divergence or adverse probabilities.

```

logspace_add <- function(x, y) {
  z <- pmax(x, y)
  z + log(exp(x - z) + exp(y - z))
}

# Exact normalizer, adverse CDF, expected loss, and KL divergence for the
# piecewise exponential tilt of a mean-one Gamma reference distribution.
gamma_tilt_components <- function(reserve_ratio, tilt, cv) {
  shape <- 1 / cv^2
  scale <- cv^2
  lower_rate <- 1 + scale * tilt * c_over
  upper_rate <- 1 - scale * tilt * c_under

  if (upper_rate <= 0) {
    stop("The tilt has crossed the Gamma moment boundary.")
  }

  lower_scale <- scale / lower_rate
  upper_scale <- scale / upper_rate
  log_lower_probability <- pgamma(
    reserve_ratio,
    shape = shape,
    scale = lower_scale,

```

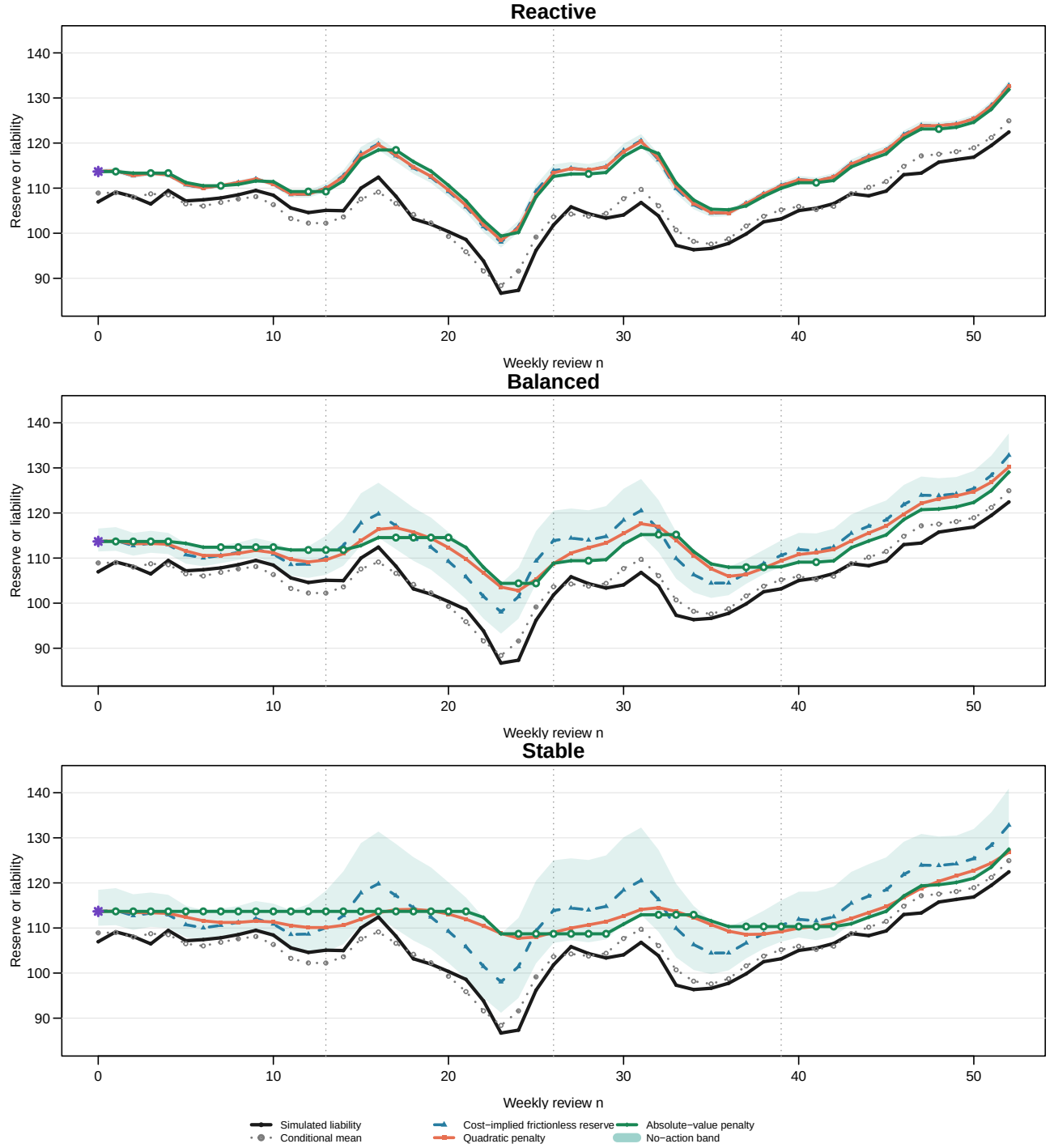


Figure 1: Nominal rolling policies under reactive, balanced, and stable revision costs. The shaded region is the absolute-value no-action band; white-centered green circles mark exact inaction decisions; dotted vertical lines mark quarter boundaries. The simulated liability is a visual reference only.

```

    log.p = TRUE
  )
  log_upper_probability <- pgamma(
    reserve_ratio,
    shape = shape,
    scale = upper_scale,
    lower.tail = FALSE,
    log.p = TRUE
  )

  log_z_lower <-
    tilt * c_over * reserve_ratio -
    shape * log(lower_rate) +
    log_lower_probability
  log_z_upper <-
    -tilt * c_under * reserve_ratio -
    shape * log(upper_rate) +
    log_upper_probability
  log_normalizer <- logspace_add(log_z_lower, log_z_upper)
  adverse_cdf <- exp(log_z_lower - log_normalizer)

  lower_first_ratio <- exp(
    pgamma(
      reserve_ratio,
      shape = shape + 1,
      scale = lower_scale,
      log.p = TRUE
    ) - log_lower_probability
  )
  upper_first_ratio <- exp(
    pgamma(
      reserve_ratio,
      shape = shape + 1,
      scale = upper_scale,
      lower.tail = FALSE,
      log.p = TRUE
    ) - log_upper_probability
  )

  lower_conditional_mean <- lower_first_ratio / lower_rate
  upper_conditional_mean <- upper_first_ratio / upper_rate
  expected_loss_ratio <-
    adverse_cdf * c_over *
    pmax(reserve_ratio - lower_conditional_mean, 0) +
    (1 - adverse_cdf) * c_under *
    pmax(upper_conditional_mean - reserve_ratio, 0)

  list(
    log_normalizer = log_normalizer,
    adverse_cdf = adverse_cdf,
    kl = tilt * expected_loss_ratio - log_normalizer,
    right_tail_scale_ratio = 1 / upper_rate
  )
}

```

The inner solver below finds the tilt that exhausts a given KL radius. The outer solver then finds the reserve whose

adverse CDF equals the required marginal probability.

```
solve_tilt_for_kl <- function(reserve_ratio, cv, epsilon) {
  if (epsilon <= 0) {
    return(0)
  }

  tilt_limit <- 1 / (cv^2 * c_under)
  high <- tilt_limit / 256

  while (
    gamma_tilt_components(reserve_ratio, high, cv)$kl < epsilon &&
    high < 0.95 * tilt_limit
  ) {
    high <- min(1.7 * high, 0.95 * tilt_limit)
  }

  if (gamma_tilt_components(reserve_ratio, high, cv)$kl < epsilon) {
    high <- (1 - 1e-10) * tilt_limit
  }

  uniroot(
    function(tilt) {
      gamma_tilt_components(reserve_ratio, tilt, cv)$kl - epsilon
    },
    interval = c(0, high),
    tol = 1e-11
  )$root
}

robust_probability_reserve_ratio <- function(
  cv,
  epsilon,
  target_probability) {

  if (epsilon <= 0) {
    return(qgamma(
      target_probability,
      shape = 1 / cv^2,
      scale = cv^2
    ))
  }

  residual <- function(reserve_ratio) {
    tilt <- solve_tilt_for_kl(reserve_ratio, cv, epsilon)
    gamma_tilt_components(reserve_ratio, tilt, cv)$adverse_cdf -
      target_probability
  }

  bracket <- qgamma(
    c(1e-7, 1 - 1e-9),
    shape = 1 / cv^2,
    scale = cv^2
  )
  uniroot(residual, interval = bracket, tol = 1e-10)$root
}
```

```

robust_probability_path <- function(
  input,
  epsilon,
  target_probability) {

  ratio <- mapply(
    robust_probability_reserve_ratio,
    cv = input$cv,
    MoreArgs = list(
      epsilon = epsilon,
      target_probability = target_probability
    )
  )
  tilt <- mapply(
    solve_tilt_for_kl,
    reserve_ratio = ratio,
    cv = input$cv,
    MoreArgs = list(epsilon = epsilon)
  )
  components <- Map(
    gamma_tilt_components,
    reserve_ratio = ratio,
    tilt = tilt,
    cv = input$cv
  )

  list(
    reserve = input$mean * ratio,
    ratio = ratio,
    tilt = tilt,
    adverse_cdf = vapply(
      components,
      function(x) x$adverse_cdf,
      numeric(1)
    ),
    kl = vapply(components, function(x) x$kl, numeric(1)),
    right_tail_scale_ratio = vapply(
      components,
      function(x) x$right_tail_scale_ratio,
      numeric(1)
    )
  )
}

```

The quadratic and absolute-value robust policies reuse the same two equations. Only their marginal probability conditions differ.

```

robust_quadratic_policy <- function(
  lambda,
  input,
  epsilon,
  robust_frictionless) {

  eta <- lambda * C_cost * input$initial
  reserve <- numeric(length(input$week))

```

```

reserve[1L] <- input$initial
foc_error <- kl_error <- numeric(length(input$week))

for (i in input$decision) {
  previous <- reserve[i - 1L]
  robust_target <- robust_frictionless$reserve[i]

  residual <- function(r) {
    ratio <- r / input$mean[i]
    tilt <- solve_tilt_for_kl(ratio, input$cv[i], epsilon)
    adverse_cdf <- gamma_tilt_components(
      ratio,
      tilt,
      input$cv[i]
    )$adverse_cdf

    C_cost * adverse_cdf - c_under +
      2 * eta * (r - previous) / previous^2
  }

  bracket <- sort(c(previous, robust_target))
  if (diff(bracket) <= 1e-12 * max(1, bracket[2L])) {
    reserve[i] <- robust_target
  } else {
    reserve[i] <- uniroot(
      residual,
      interval = bracket,
      tol = 1e-10
    )$root
  }

  ratio <- reserve[i] / input$mean[i]
  tilt <- solve_tilt_for_kl(ratio, input$cv[i], epsilon)
  component <- gamma_tilt_components(ratio, tilt, input$cv[i])
  foc_error[i] <- residual(reserve[i])
  kl_error[i] <- component$kl - epsilon
}

list(
  reserve = reserve,
  eta = eta,
  max_foc_error = max(abs(foc_error[input$decision])),
  max_kl_error = max(abs(kl_error[input$decision]))
)
}

robust_absolute_policy <- function(eta, input, epsilon) {
  lower_probability <- q_star - eta / C_cost
  upper_probability <- q_star + eta / C_cost
  lower_solution <- robust_probability_path(
    input,
    epsilon,
    lower_probability
  )
  upper_solution <- robust_probability_path(

```

```

    input,
    epsilon,
    upper_probability
  )

  reserve <- numeric(length(input$week))
  reserve[1L] <- input$initial
  action <- rep("Initial", length(input$week))

  for (i in input$decision) {
    previous <- reserve[i - 1L]
    lower <- lower_solution$reserve[i]
    upper <- upper_solution$reserve[i]

    if (previous < lower) {
      reserve[i] <- lower
      action[i] <- "Increase"
    } else if (previous > upper) {
      reserve[i] <- upper
      action[i] <- "Decrease"
    } else {
      reserve[i] <- previous
      action[i] <- "Keep"
    }
  }

  list(
    reserve = reserve,
    lower = lower_solution$reserve,
    upper = upper_solution$reserve,
    action = action,
    lower_solution = lower_solution,
    upper_solution = upper_solution,
    max_kl_error = max(abs(c(
      lower_solution$kl[input$decision] - epsilon,
      upper_solution$kl[input$decision] - epsilon
    ))),
    max_probability_error = max(abs(c(
      lower_solution$adverse_cdf[input$decision] - lower_probability,
      upper_solution$adverse_cdf[input$decision] - upper_probability
    )))
  )
}

```

We now hold the balanced revision parameters fixed and solve all three policies at each KL radius.

```

balanced_spec <- subset(
  scenario_spec,
  Configuration == "Balanced"
)
balanced_nominal <- nominal_paths[["Balanced"]]

robust_results <- lapply(
  seq_len(nrow(ambiguity_spec)),
  function(j) {
    epsilon <- ambiguity_spec$Epsilon[j]

```

```

robust_frictionless <- robust_probability_path(
  input,
  epsilon,
  q_star
)
robust_frictionless$reserve[1L] <- input$initial

robust_quadratic <- robust_quadratic_policy(
  balanced_spec$QuadraticLambda,
  input,
  epsilon,
  robust_frictionless
)
robust_absolute <- robust_absolute_policy(
  balanced_spec$EtaAbsolute,
  input,
  epsilon
)

policies <- list(
  "Frictionless reserve" = list(
    reserve = robust_frictionless$reserve,
    nominal = input$frictionless,
    action = NULL
  ),
  "Quadratic penalty" = list(
    reserve = robust_quadratic$reserve,
    nominal = balanced_nominal$Quadratic,
    action = NULL
  ),
  "Absolute-value penalty" = list(
    reserve = robust_absolute$reserve,
    nominal = balanced_nominal$Absolute,
    action = robust_absolute$action
  )
)

metrics <- lapply(names(policies), function(policy_name) {
  object <- policies[[policy_name]]
  row <- evaluate_path(
    object$reserve,
    input,
    ambiguity_spec$Ambiguity[j],
    policy_name,
    action = object$action
  )
  row$Epsilon <- epsilon
  row$UpliftVsNominalPct <- 100 * mean(
    object$reserve[input$decision] /
    object$nominal[input$decision] - 1
  )
  row
})

d <- input$decision

```

```

diagnostics <- data.frame(
  Ambiguity = ambiguity_spec$Ambiguity[j],
  Epsilon = epsilon,
  FrictionlessKLError = max(abs(
    robust_frictionless$kl[d] - epsilon
  )),
  FrictionlessProbabilityError = max(abs(
    robust_frictionless$adverse_cdf[d] - q_star
  )),
  QuadraticKLError = robust_quadratic$max_kl_error,
  QuadraticFOCError = robust_quadratic$max_foc_error,
  AbsoluteKLError = robust_absolute$max_kl_error,
  AbsoluteProbabilityError =
    robust_absolute$max_probability_error,
  stringsAsFactors = FALSE
)

list(
  frictionless = robust_frictionless,
  quadratic = robust_quadratic,
  absolute = robust_absolute,
  metrics = do.call(rbind, metrics),
  diagnostics = diagnostics
)
}
)
names(robust_results) <- ambiguity_spec$Ambiguity

robust_metrics <- do.call(
  rbind,
  lapply(robust_results, function(x) x$metrics)
)
robust_diagnostics <- do.call(
  rbind,
  lapply(robust_results, function(x) x$diagnostics)
)
rownames(robust_metrics) <- NULL
rownames(robust_diagnostics) <- NULL

```

Table 3: KL-robust policies with balanced revision costs held fixed.

Ambiguity	Epsilon	Policy	Uplift (%)	Regret (%)	Avg. percentile	Movement reduction (%)
Small	0.005	Frictionless reserve	0.51	0.11	76.46	-5.60
Small	0.005	Quadratic penalty	0.50	1.40	75.65	24.37
Small	0.005	Absolute-value penalty	0.47	3.55	74.79	47.27
Moderate	0.080	Frictionless reserve	2.04	1.78	80.54	-9.88
Moderate	0.080	Quadratic penalty	2.02	2.53	79.77	19.36
Moderate	0.080	Absolute-value penalty	1.90	3.60	78.58	41.16
Large	0.250	Frictionless reserve	3.63	5.39	84.24	-14.30
Large	0.250	Quadratic penalty	3.60	5.71	83.53	15.22

Ambiguity	Epsilon	Policy	Uplift (%)	Regret (%)	Avg. percentile	Movement reduction (%)
Large	0.250	Absolute-value penalty	3.46	5.99	82.37	35.95

Increasing ε raises the reserve level but does not mechanically alter responsiveness. At the largest radius, mean uplift relative to the corresponding nominal policies is approximately 3.5–3.6%. The attained nominal percentile rises because the robust law allocates more weight to costly shortfall outcomes. The quadratic and absolute-value policies still display gradual adjustment and inaction, respectively.

5.1 Robust reserve paths

```
robust_colors <- c(
  Small = "#56B4E9",
  Moderate = "#6F42C1",
  Large = "#B2182B"
)

draw_robust_panel <- function(policy_key, panel_title) {
  nominal_series <- switch(
    policy_key,
    frictionless = input$frictionless,
    quadratic = balanced_nominal$Quadratic,
    absolute = balanced_nominal$Absolute
  )
  robust_series <- lapply(robust_results, function(x) {
    x[[policy_key]]$reserve
  })
  policy_color <- switch(
    policy_key,
    frictionless = plot_colors["frictionless"],
    quadratic = plot_colors["quadratic"],
    absolute = plot_colors["absolute"]
  )
  policy_pch <- switch(
    policy_key,
    frictionless = 17,
    quadratic = 15,
    absolute = 18
  )
  limits <- range(
    input$mean,
    simulated_liability,
    nominal_series,
    unlist(robust_series)
  )
  limits <- limits + c(-0.05, 0.05) * diff(limits)

  plot(
    input$week,
    nominal_series,
    type = "n",
    xlim = c(0, N),
```

```

ylim = limits,
xlab = "Weekly review n",
ylab = "Reserve or liability",
las = 1
)
abline(h = pretty(limits), col = "#E4E4E4", lwd = 0.7)
abline(v = c(13, 26, 39), col = "#999999", lty = 3)
lines(
  input$week,
  simulated_liability,
  col = plot_colors["simulated"], lwd = 2.6
)
points(
  input$week, simulated_liability,
  col = plot_colors["simulated"], pch = 16, cex = 0.58
)
lines(input$week, input$mean, col = plot_colors["mean"], lty = 3, lwd = 1.8)
points(
  input$week, input$mean,
  col = plot_colors["mean"], pch = 1, cex = 0.62
)
lines(input$week, nominal_series, col = policy_color, lty = 2, lwd = 2.2)
points(
  input$week, nominal_series,
  col = policy_color, pch = policy_pch, cex = 0.66
)
}
if (policy_key == "absolute") {
  nominal_keep <- balanced_nominal$AbsoluteAction == "Keep" &
    input$week > 0
  points(
    input$week[nominal_keep], nominal_series[nominal_keep],
    pch = 21, bg = "white", col = plot_colors["absolute"],
    lwd = 1.6, cex = 0.84
  )
}

for (ambiguity in names(robust_results)) {
  lines(
    input$week,
    robust_results[[ambiguity]][[policy_key]]$reserve,
    col = robust_colors[ambiguity],
    lwd = 2
  )
  points(
    input$week,
    robust_results[[ambiguity]][[policy_key]]$reserve,
    col = robust_colors[ambiguity], pch = 15, cex = 0.52
  )
  if (policy_key == "absolute") {
    robust_keep <- robust_results[[ambiguity]]$absolute$action == "Keep" &
      input$week > 0
    points(
      input$week[robust_keep],
      robust_results[[ambiguity]]$absolute$reserve[robust_keep],
      pch = 21, bg = "white", col = robust_colors[ambiguity],
      lwd = 1.5, cex = 0.80
    )
  }
}

```

```

    )
  }
}
points(
  0, input$initial, pch = 8,
  col = plot_colors["initial"], lwd = 2, cex = 1.05
)
mtext(panel_title, side = 3, line = 0.25, font = 2)
}

```

6 Predictive-dispersion sensitivity

To test whether the qualitative comparison is an artifact of the chosen CV sequence, the balanced policies are rerun at $0.75v_n$, v_n , and $1.25v_n$, with the mean sequence unchanged.

```

cv_sensitivity <- do.call(
  rbind,
  lapply(c(0.75, 1, 1.25), function(multiplier) {
    local_input <- make_input(multiplier)
    local_input$initial <- local_input$frictionless[1L]
    quadratic <- quadratic_policy(
      balanced_spec$QuadraticLambda,
      local_input
    )
    absolute <- absolute_policy(
      balanced_spec$EtaAbsolute,
      local_input
    )

    quadratic_metrics <- evaluate_path(
      quadratic$reserve,
      local_input,
      "Balanced",
      "Quadratic penalty"
    )
    absolute_metrics <- evaluate_path(
      absolute$reserve,
      local_input,
      "Balanced",
      "Absolute-value penalty",
      action = absolute$action
    )

    data.frame(
      CVMultiplier = multiplier,
      QuadraticRegret = quadratic_metrics$RegretPct,
      AbsoluteRegret = absolute_metrics$RegretPct,
      QuadraticMovementReduction =
        quadratic_metrics$MovementReductionVsMeanPct,
      AbsoluteInaction = absolute_metrics$InactionPct
    )
  })
)

```

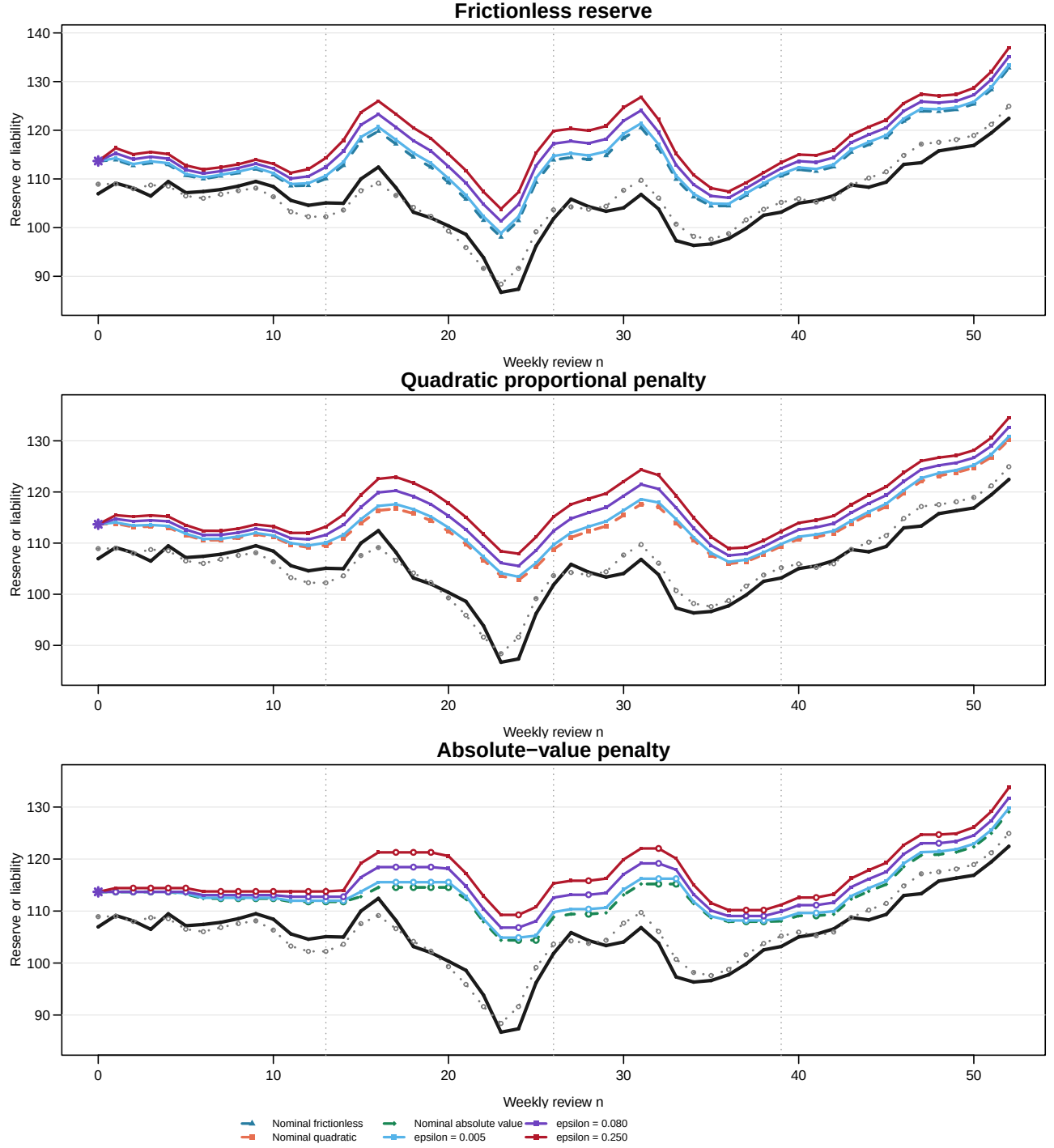


Figure 2: KL sensitivity by policy. Balanced revision costs are fixed in every panel; only the ambiguity radius changes. White-centered circles mark exact inaction decisions in the absolute-value panel.

Table 4: Balanced-policy sensitivity to predictive dispersion.

CV multiplier	Quadratic regret (%)	Absolute regret (%)	Quadratic movement reduction (%)	Absolute inaction (%)
0.75	1.79	4.19	21.98	36.54
1.00	1.49	4.05	26.34	44.23
1.25	1.27	3.76	30.13	55.77

The ordering is unchanged. Higher predictive dispersion modestly increases smoothing and widens the absolute-value no-action region, while adequacy regret remains close to its baseline value.

7 Numerical verification

Every reported policy is checked against the equation that defines it. The largest residual across the nominal and robust calculations is reported below; values near machine precision confirm that the figures are not artifacts of loose numerical tolerances.

```

diagnostic_values <- c(
  nominal_diagnostics$MaxQuadraticFOCError,
  unlist(robust_diagnostics[-c(1, 2)])
)

diagnostic_summary <- data.frame(
  Check = c(
    "Largest nominal quadratic FOC residual",
    "Largest robust-system residual"
  ),
  MaximumAbsoluteError = c(
    max(nominal_diagnostics$MaxQuadraticFOCError),
    max(unlist(robust_diagnostics[-c(1, 2)]))
  )
)

stopifnot(
  all(is.finite(diagnostic_values)),
  max(diagnostic_values) < 1e-8,
  all(diff(input$week) == 1),
  length(input$week) == N + 1L,
  all(vapply(nominal_paths, nrow, integer(1)) == N + 1L)
)

```

Table 5: Numerical residual checks.

Check	MaximumAbsoluteError
Largest nominal quadratic FOC residual	9.10e-13
Largest robust-system residual	1.33e-10

8 Conclusion

The controlled experiment isolates the decision layer from model estimation. The frictionless rule converts asymmetric adequacy costs into an economically motivated reserve buffer. The quadratic penalty exchanges a

small amount of adequacy for gradual adjustment, while the absolute-value penalty creates literal inaction periods and discrete interventions. Moving from reactive to stable revision costs strengthens both effects in the expected direction.

The balanced configuration is useful as a middle reference because it remains responsive to the middle-year reversal and late trend while reducing revision activity. It is not a universal recommendation: a practitioner should calibrate the cost asymmetry and revision penalties to its own capital, liquidity, governance, and implementation environment.

KL robustness answers a different question. It raises the reserve against predictive model uncertainty while leaving the selected revision-cost regime fixed. The simulation therefore confirms the intended separation: robustness changes the predictive measure or reserve level, whereas revision penalties control how the booked reserve responds from one review to the next.

#> Report generated with R version 4.1.1 (2021-08-10).